# RESEARCH podman_k8s

# RESEARCH podman_k8s

Scope: Rootless Podman quadlets (systemd `.container`/`.pod`) for a Mullvad-parity self-hosted VPN edge — `AddCapability=NET_ADMIN`, read-only rootfs + seccomp, `:443/udp` exposure, a single pod running Postgres + Redis + app; the same workload mapped to Docker Compose and to a Kubernetes manifest set (Deployment/StatefulSet/Service/NetworkPolicy); rootless-networking caveats for a tunnel. Access date for every source below: 2026-06-25. Web access: YES.

NOTE on dates: WebSearch reports "current month is June 2026"; several cited blog posts carry 2026-03 datelines. Authoritative primary sources are the upstream Podman docs, Kubernetes docs, Docker docs, and the containers/* GitHub repos.

---

## 1. Quadlet fundamentals (current, Podman 5.x)

Quadlet is now a **core part of Podman** (not a separate project); Podman 5.x added the unit-file commands `podman quadlet list|print|install|rm`, and new unit types `.pod`, `.build`, `.image`, `.artifact` beyond the original `.container`/`.network`/`.volume`/`.kube`. Quadlet `.unit` files are read by a systemd generator that emits real `.service` units at boot/daemon-reload. [podman-systemd.unit man page; Red Hat "Make systemd better for Podman with Quadlet"; Podman Desktop blog]

### Rootless unit search paths (verbatim from upstream man page)

A rootless quadlet is selected purely by **where the file lives** — there is no `User=` switch:

- `$XDG_RUNTIME_DIR/containers/systemd/`
- `$XDG_CONFIG_HOME/containers/systemd/` (i.e. `~/.config/containers/systemd/`)

- `/etc/containers/systemd/users/${UID}`
- `/etc/containers/systemd/users/`
- `/usr/share/containers/systemd/users/${UID}` and `/usr/share/containers/systemd/users/`

**Hard constraint:** "Quadlet units do not support running as a non-root user by defining the User, Group, or DynamicUser systemd options." To run rootless you MUST place the unit in one of the rootless search paths above and (for boot-time start without an interactive login) enable `loginctl enable-linger <user>`. [https://docs.podman.io/en/latest/markdown/podman-systemd.unit.5.html]

## `[Container]` directives relevant to a VPN edge (names verbatim)

- `Image=` (required; use a fully-qualified name).
- `AddCapability=` — "Add these capabilities, in addition to the default Podman capability set, to the container." Space-separated, multiple entries allowed. `DropCapability=` is the inverse. For a VPN edge: `DropCapability=ALL` then `AddCapability=NET_ADMIN NET_RAW`.
- `AddDevice=` — adds host device nodes (e.g. `AddDevice=/dev/net/tun:/dev/net/tun` for a userspace/kernel tunnel).
- `PublishPort=` — "Exposes a port, or a range of ports … from the container to the host" (e.g. `PublishPort=443:443/udp`).
- `Network=` — custom network; a `name.network` value creates a dependency on a `.network` quadlet.
- `Pod=` — "Specify a Quadlet `.pod` unit to link the container to" (value form `name.pod`; Quadlet auto-wires the ordering between the pod service and the member-container services).
- `Volume=` — supports `name.volume` references with auto-generated dependencies.
- `ReadOnly=` (default false) — "makes the image read-only" (read-only rootfs).
- `ReadOnlyTmpfs=` (default true) — when `ReadOnly=true`, mounts rw tmpfs on `/dev`, `/dev/shm`, `/run`, `/tmp`, `/var/tmp`.
- `SeccompProfile=` — "Set the seccomp profile … If unset, the default podman profile is used."
- `SecurityLabelDisable=` — turns off SELinux label separation (avoid unless necessary).
- `Environment=` / `EnvironmentFile=`, `Notify=` (sd_notify passthrough), `HealthCmd=`. [https://docs.podman.io/en/latest/markdown/podman-systemd.unit.5.html]

### `[Pod]` directives

A `.pod` file carries a `[Pod]` section; pod-level `PublishPort=`, `Network=`, `Volume=` apply to the whole pod (containers in a pod share the network namespace, so ports are published at the POD, not the member container). The generated pod service defaults to `Type=forking` / `Restart=on-failure`. The generator guarantees the `*-pod.service` starts before any member `Pod=foo.pod` container. `[Install]` keys (`WantedBy`, `RequiredBy`, `Alias`, `UpheldBy`) pass through. [podman-systemd.unit man page; Oracle Linux "9 Podman Quadlets"; Podman Desktop blog]

## Example skeleton — one pod, Postgres+Redis+app, VPN edge container

```
# ~/.config/containers/systemd/helix.pod
[Pod]
PublishPort=443:443/udp
# pod-wide network; member containers reach each other on
        127.0.0.1

# ~/.config/containers/systemd/vpn-edge.container
[Container]
Image=ghcr.io/example/helix-vpn-edge:1.0
Pod=helix.pod
DropCapability=ALL
AddCapability=NET_ADMIN NET_RAW
AddDevice=/dev/net/tun:/dev/net/tun
ReadOnly=true
ReadOnlyTmpfs=true
SeccompProfile=/etc/containers/seccomp/vpn.json
Notify=true
[Service]
Restart=always
[Install]
WantedBy=default.target

# postgres.container / redis.container similarly carry
        Pod=helix.pod,
# DropCapability=ALL, ReadOnly=true with a named Volume= for data.
```

Place all three `.container` files + the `.pod` in the same rootless search dir; `systemctl --user daemon-reload && systemctl --user start helix-pod.service`. [synthesized from man page directives]

---

# 2. Capabilities for a tunnel: NET_ADMIN is necessary but not sufficient

- Kernel-mode **WireGuard** runs in rootless Podman, but you must add BOTH `--cap-add NET_ADMIN` AND `--cap-add NET_RAW` — "WireGuard needs NET_RAW … which Docker enables by default but Podman does not." (Podman's default cap set already includes NET_RAW for root containers but the rootless default set is reduced; explicitly add it.)
- The **WireGuard kernel module must be loaded on the host, outside Podman** — all kernels ⩾ 5.6 ship it built-in, but a container cannot `modprobe` it rootlessly.
- For a TUN-based (userspace, e.g. wireguard-go / OpenVPN / boringtun) tunnel, add `AddDevice=/dev/net/tun:/dev/net/tun` plus `NET_ADMIN`. [https://www.procustodibus.com/blog/2022/10/wireguard-in-podman/ ; https://emar10.dev/posts/rootless-podman-wireguard/ ; https://oneuptime.com/blog/post/2026-03-18-use-podman-containers-wireguard-vpn/view]

---

# 3. Rootless networking caveats (the load-bearing section for a VPN)

## Backend: pasta is now the default (replaces slirp4netns)

- Podman 5.x defaults the rootless network backend to **pasta**; from **RHEL 9.5 onward pasta is the default** rootless network mode. Verify with `podman info` → `host.networkBackend` / `rootlessNetworkCmd` (shows `pasta` or `slirp4netns`).
- pasta "copies the host's network configuration into the container" and **avoids NAT**, giving better throughput than slirp4netns, which "translates every single TCP/UDP packet into a syscall on the host" and "struggles to scale with multi-core."
- CAVEAT (regression to watch): an open upstream issue reports **pasta throughput regressions in Podman 5.8** for some workloads — benchmark your build, don't assume. [https://sanj.dev/post/podman-pasta-vs-slirp4netns-networking/ ; https://docs.oracle.com/en/learn/ol-podman-pasta-networking/ ; https://github.com/containers/podman/issues/28219 ; https://github.com/eriksjolund/podman-networking-docs]

### Firewall / NAT not auto-managed rootlessly

- "Modifying the firewall requires root access … so rootless Podman does not [set up iptables rules]." A VPN edge that needs to NAT/forward client traffic out the host cannot rely on Podman to install those rules — you provide host-side forwarding (or run the masquerade inside the container's own netns with NET_ADMIN).
- MTU pitfall: tunnels frequently break ("connected but nothing loads") because of MTU; adjust the podman network / WireGuard MTU.
- slirp4netns offers `--outbound-addr=[IPv4|INTERFACE]` to pin the source IP/interface for outbound packets (relevant when the VPN must egress a specific uplink). [https://oneuptime.com/blog/post/2026-03-18-use-podman-containers-wireguard-vpn/view ; https://github.com/wg-easy/wg-easy/wiki/Using-WireGuard-Easy-with-rootless-Podman-(incl.-Kubernetes-yaml-file-generation)]

### Binding `:443` rootlessly

Rootless processes cannot bind ports < 1024 by default (`net.ipv4.ip_unprivileged_port_start=1024`). Two current options:

1. **Lower the sysctl** (most common): `sysctl -w net.ipv4.ip_unprivileged_port_start=443`, persisted in `/etc/sysctl.d/99-podman-privileged-ports.conf` — then `PublishPort=443:...` works rootlessly.
2. Front the edge with a root-side proxy / `authbind` / port-forward. For `:443/udp` (QUIC/WireGuard- style), the sysctl approach is the clean one. [https://oneuptime.com/blog/post/2026-03-18-bind-privileged-ports-rootless-podman/view ; https://oneuptime.com/blog/post/2026-03-18-configure-ip-unprivileged-port-start-rootless-podman/view ; https://github.com/containers/podman/blob/main/rootless.md]

---

## 4. Same workload → Docker Compose

Compose `service` keys mapping the quadlet directives:

- `cap_drop: ["ALL"]` + `cap_add: ["NET_ADMIN","NET_RAW"]` (least-privilege baseline: drop all, add only what's needed).
- `devices: ["/dev/net/tun:/dev/net/tun"]` for the TUN device (VPN/tunnel apps).
- `read_only: true` for a read-only rootfs; writable paths via `tmpfs: [/run, /tmp]`.
- `security_opt: ["seccomp=./vpn-seccomp.json"]` (or `seccomp=unconfined` to disable; Docker's default profile blocks ~44 syscalls).

- `ports: ["443:443/udp"]`.
- A "single pod" maps to **one Compose project / shared network**; Compose has no pod primitive, so inter-service traffic uses the project network (service DNS names) rather than `127.0.0.1`.
- `sysctls:` can set per-container kernel params; host-level `net.ipv4.ip_unprivileged_port_start` still governs rootless-Docker privileged binds. [https://docs.docker.com/engine/security/seccomp/ ; https://lours.me/posts/compose-tip-029-container-capabilities/ ; https://oneuptime.com/blog/post/2026-01-25-docker-container-capabilities/view ; https://oneuptime.com/blog/post/2026-01-30-docker-security-context/view]

Podman can also **consume the pod as Kubernetes YAML** directly: `podman kube play` runs a K8s Pod/Deployment YAML, and `podman kube generate` emits one from running containers — a bridge between the three formats. [podman-quadlet / kube docs; wg-easy wiki above]

---

# 5. Same workload → Kubernetes manifest set

### Per-container `securityContext` (maps the quadlet security directives)

```yaml
securityContext:
  readOnlyRootFilesystem: true            # == ReadOnly=true
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]                          # == DropCapability=ALL
    add: ["NET_ADMIN","NET_RAW"]           # ==
        AddCapability=NET_ADMIN NET_RAW
  seccompProfile:
    type: RuntimeDefault                   # == default podman/
        seccomp; or type: Localhost + localhostProfile
```

- `seccompProfile.type` ∈ `RuntimeDefault | Unconfined | Localhost`. `RuntimeDefault` = runtime's default profile (minimal syscall set). For a custom profile use `Localhost` + `localhostProfile: <path>` (file under the kubelet seccomp dir). `--seccomp-default` makes RuntimeDefault the cluster baseline.
- TUN device: Kubernetes has **no first-class `/dev/net/tun` mount**; options are a `privileged` init/sidecar, a device-plugin, or `hostPath` of `/dev/net/tun` — each weakens isolation; document the trade-off. (This is where the quadlet `AddDevice=` is strictly simpler than vanilla K8s.) [https://kubernetes.io/docs/tasks/configure-pod-container/security-context/ ; https://oneuptime.com/blog/post/2026-02-09-capabilities-drop-all-add-specific/view]

### Workload objects

- **app / VPN edge** → `Deployment` (stateless replicas) — or a `DaemonSet` if the edge must bind a host interface per node.
- **Postgres** → `StatefulSet` (stable network id + `volumeClaimTemplates` for the PVC) — NOT a Deployment (data + stable identity).
- **Redis** → `Deployment` for a simple cache, or `StatefulSet` if persistence/replication matters.
- **Service** → the VPN edge `:443/udp` is exposed via `Service type: LoadBalancer` (or `NodePort` / `hostPort` on the pod for a direct edge); set `protocol: UDP` on the port. Postgres and Redis get **headless** `Service` (`clusterIP: None`) for stable DNS within the namespace.
- A K8s **Pod can hold all containers** (sidecar pattern) to mirror the single-pod quadlet, but the idiomatic split is separate workloads + Services.

### NetworkPolicy (the K8s analogue of "the pod is the only ingress")

- Default-deny ingress/egress, then allow only: client→edge `:443/udp`, edge→postgres `:5432/tcp`, edge→redis `:6379/tcp`, and required DNS egress `:53`. NetworkPolicy is namespaced and additive (allow-only); it requires a CNI that enforces it (Calico/Cilium/etc.). [https://kubernetes.io/docs/tasks/configure-pod-container/security-context/ — securityContext; standard K8s workload/Service/NetworkPolicy semantics]

---

## 6. Cross-format mapping table (quick reference)

| Concern | Quadlet (`[Container]`/`[Pod]`) | Docker Compose | Kubernetes |
|---|---|---|---|
| Add capability | `AddCapability=NET_ADMIN NET_RAW` | `cap_add: [NET_ADMIN, NET_RAW]` | `securityContext.capa` |
| Drop all caps | `DropCapability=ALL` | `cap_drop: [ALL]` | `capabilities.drop: [` |
| Read-only rootfs | `ReadOnly=true` (+`ReadOnlyTmpfs`) | `read_only: true` (+`tmpfs:`) | `readOnlyRootFilesyst` |
| Seccomp | `SeccompProfile=<path>` | `security_opt: [seccomp=<path>]` | `seccompProfile.type: RuntimeDefault\|Local` |

| Concern | Quadlet (`[Container]`/`[Pod]`) | Docker Compose | Kubernetes |
|---|---|---|---|
| TUN device | `AddDevice=/dev/net/tun:/dev/net/tun` | `devices: [/dev/net/tun:/dev/net/tun]` | hostPath `/dev/net/tun` plugin / privileged |
| Publish `:443/udp` | `PublishPort=443:443/udp` | `ports: ["443:443/udp"]` | `Service` port `protocol` `hostPort` |
| Grouping | `.pod` + `Pod=helix.pod` | one project / shared network | one Pod (sidecars) or se workloads+Services |
| Stateful DB | `Volume=pgdata.volume` on container | named `volumes:` | `StatefulSet` + `volumeClaimTemplates` |
| Ingress restriction | pod is sole published surface | only `ports:` published | `NetworkPolicy` default allow-list |
| Rootless `:443` | host `ip_unprivileged_port_start=443` | same host sysctl (rootless Docker) | not applicable (kube-p binds) |

---

# 7. Key facts to carry into the spec (anti-bluff, FACT-grade)

1. Rootless quadlet = file location, never `User=`; needs `enable-linger` for boot start. [man page]
2. NET_ADMIN alone is insufficient for WireGuard/tunnels — also NET_RAW, the host WG module, and `/dev/net/tun` for userspace tunnels. [Pro Custodibus; emar10.dev]
3. pasta is the current rootless default and is NAT-free/faster than slirp4netns, BUT a 5.8 throughput regression is reported — benchmark. [sanj.dev; podman#28219]
4. Rootless Podman does NOT auto-install firewall/NAT rules; plan host-side forwarding or in-netns masquerade for a VPN that routes client egress. [oneuptime WG post]
5. Binding `:443` rootlessly = lower `net.ipv4.ip_unprivileged_port_start` (persist in sysctl.d).
6. `podman kube play` / `podman kube generate` bridge quadlet/Compose ? Kubernetes YAML directly.
7. Postgres → StatefulSet (never Deployment); edge `:443/udp` Service needs explicit `protocol: UDP`; lock the topology with a default-deny NetworkPolicy (CNI must enforce). [k8s docs]

---

# Sources verified


- https://docs.podman.io/en/latest/markdown/podman-systemd.unit.5.html — accessed 2026-06-25
- https://docs.podman.io/en/latest/markdown/podman-quadlet.1.html — accessed 2026-06-25
- https://github.com/containers/podman/blob/main/rootless.md — accessed 2026-06-25
- https://github.com/containers/podman/issues/28219 (pasta 5.8 throughput regression) — accessed 2026-06-25
- https://www.redhat.com/en/blog/quadlet-podman — accessed 2026-06-25
- https://podman-desktop.io/blog/podman-quadlet — accessed 2026-06-25
- https://docs.oracle.com/en/operating-systems/oracle-linux/podman/quadlets.html — accessed 2026-06-25
- https://docs.oracle.com/en/learn/ol-podman-pasta-networking/ — accessed 2026-06-25
- https://sanj.dev/post/podman-pasta-vs-slirp4netns-networking/ — accessed 2026-06-25
- https://github.com/eriksjolund/podman-networking-docs — accessed 2026-06-25
- https://www.procustodibus.com/blog/2022/10/wireguard-in-podman/ — accessed 2026-06-25
- https://emar10.dev/posts/rootless-podman-wireguard/ — accessed 2026-06-25
- https://github.com/wg-easy/wg-easy/wiki/Using-WireGuard-Easy-with-rootless-Podman-(incl.-Kubernetes-yaml-file-generation) — accessed 2026-06-25
- https://oneuptime.com/blog/post/2026-03-18-use-podman-containers-wireguard-vpn/view — accessed 2026-06-25
- https://oneuptime.com/blog/post/2026-03-18-bind-privileged-ports-rootless-podman/view — accessed 2026-06-25
- https://oneuptime.com/blog/post/2026-03-18-configure-ip-unprivileged-port-start-rootless-podman/view — accessed 2026-06-25
- https://kubernetes.io/docs/tasks/configure-pod-container/security-context/ — accessed 2026-06-25
- https://oneuptime.com/blog/post/2026-02-09-capabilities-drop-all-add-specific/view — accessed 2026-06-25
- https://docs.docker.com/engine/security/seccomp/ — accessed 2026-06-25
- https://lours.me/posts/compose-tip-029-container-capabilities/ — accessed 2026-06-25
- https://oneuptime.com/blog/post/2026-01-30-docker-security-context/view — accessed 2026-06-25

- https://oneuptime.com/blog/post/2026-01-25-docker-container-capabilities/view — accessed 2026-06-25